

پیش از پاسخ‌گویی دقت نمایید

- با توجه به این که امکان پاسخ‌گویی به سؤالات شما در طول امتحان وجود ندارد، اگر در یک پرسش به ابهامی برخوردید فرض معقولی انجام دهید و به پرسش پاسخ دهید. فرض انجام‌شده را در پاسخ‌نامه بنویسید.
- در پاسخ‌گویی به پرسش‌ها استفاده از تمام منابع (شامل ویدیوها و اسلایدهای درس، کتاب‌ها و منابع برخط) آزاد است به جز مشورت با افراد دیگر (شامل دانشجویان این کلاس یا افراد دیگر).
- پاسخ پرسش‌ها را در قالب یک فایل پی‌دی‌اف (تایپ‌شده یا تصویر دست‌نویس) در محل مشخص‌شده با عنوان بارگذاری پاسخ امتحان پایان ترم در سامانه ای‌لرن بارگذاری نمایید.

پرسش ۱ (۲ نمره) – جاهای خالی را در جملات زیر پر کنید.

(الف) استفاده از چندریختی انعطاف‌پذیری برنامه را می‌دهد و/اما به خوانایی آن لطمه

(ب) تایپ کلاس‌های مجرد (abstract) فقط به شکل قابل استفاده هستند.

(پ) برای کلاسی که در خود تخصیص حافظه‌ی پویا دارد لازم است و را تعریف کنیم.

(ت) لیست‌های پیوندی برای عملیات و سریع‌تر و برای کندتر عمل می‌کنند.

با توجه به توضیحات زیر به سؤالاتی که در صفحه بعد پاسخ دهید

کلاس Node مطابق آنچه در زیر تعریف شده است، معادل یک گره از یک درخت است که علاوه بر نام آن گره، برادری از اشاره‌گر به فرزندان آن گره نگه می‌دارد.

```
class Node {
public:
    Node(string n) : name(n) {}

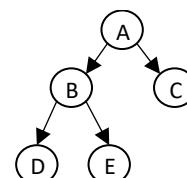
    void add_child(string n) {
        children.push_back(new Node(n));
    }

    Node* get_child(string n) {
        for (int i = 0; i < children.size(); i++)
            if (children[i]->name == n)
                return children[i];
        return NULL;
    }

    string get_name() const { return name; }
private:
    string name;
    vector<Node*> children;
};
```

به عنوان مثال، قطعه کد زیر (سمت چپ)، درختی معادل شکل زیر (سمت راست) ایجاد می‌کند:

```
Node root("A");
root.add_child("B");
root.add_child("C");
root.get_child("B")->add_child("D");
root.get_child("B")->add_child("E");
```



پرسش ۲ (۱ نمره) – وجود const بعد از امضای متد get_name به چه معنی است؟ آیا در مثال بالا، می‌توان فراخوانی به صورت root.get_name() را داشت یا خطا پیش می‌آید؟

پرسش ۳ (۱ نمره) – اگر بخواهیم مقدار ذخیره‌شده در گره‌ها به جای string بتواند از تایپ‌های دیگر هم باشد از چه امکانی از زبان ++C استفاده می‌کنیم؟ (لازم نیست کلاس تغییر یافته را بنویسید).

پرسش ۴ (۱ نمره) – چرا نمی‌توانیم تعریف فرزندان یک گره را به صورت vector<Node> children انجام دهیم؟

پرسش ۵ (۳ نمره) – متدی به نام print به این کلاس اضافه کنید که هر گاه روی یک گره صدا شود ابتدا نام آن گره را می‌نویسد و بعد در داخل دو علامت [] تمام فرزندان آن را (به طور بازگشتی) print می‌کند. در مثال بالا root.print() در خروجی A [B [D [] E []] C [] را می‌نویسد.

پرسش ۶ (۳ نمره) – برای این کلاس مخرب (destructor) و سازنده کپی (copy constructor) بنویسید.

پرسش ۷ (۲ نمره) – عملگر () را برای کلاس بازنویسی کنید طوری که همان کار get_child را انجام دهد. به عنوان مثال، root("C") معادل است با root.get_child("C").

پرسش ۸ (۲ نمره) – با فرض داشتن تابع find_if در کتابخانه‌ی استاندارد که به شکل زیر تعریف شده، بدنه‌ی get_child را با استفاده از find_if پیاده‌سازی کنید. راهنمایی: برای این کار از شیء تابع (function object) استفاده کنید.

```
template<class In, class Pred>
In find_if(In first, In last, Pred pred) {
    while (first!=last && !pred(*first))
        ++first;
    return first;
}
```

پرسش ۹ (۲ نمره) – متدی با امضای Node::preorder() برای کلاس Node تعریف کنید که یک بردار از رشته‌ها برمی‌گرداند که تمام عناصر درخت را به طور پیش‌ترتیب در خود ذخیره کرده است (پیش‌ترتیب یعنی اول ریشه، بعد هر یک از زیردرخت‌های آن به طور پیش‌ترتیب). در مثال بالا، این بردار مانند ["C", "E", "D", "B", "A"] خواهد بود. برای تعریف این متد می‌توانید متدهای کمکی دیگری نیز تعریف کنید.

پرسش ۱۰ (۳ نمره) – در این پرسش می‌خواهیم یک iterator برای این درخت بنویسیم. بخش‌های زیر را داخل کلاس Node می‌نویسیم:

```
class Iterator : public std::iterator<input_iterator_tag, string> {
public:
    (1) // constructor
    Iterator& operator++() {
        (2)
        return *this;
    }
    string operator*() { (3) }
    bool operator==(const Iterator& it) { (4) }
    bool operator!=(const Iterator& it) { (5) }
private:
    (6)
};

Iterator begin() { (7) }
Iterator end() { (8) }
```

این iterator هنگام ساخته شدن، با استفاده از متد preorder پرسش قبل عناصر درخت را در یک بردار نگه می‌دارد و به کمک آن امکان پیمایش عناصر با iterator را فراهم می‌سازد. می‌توانید فرض کنید پس از ساخت یک iterator عناصر درخت تغییر نمی‌کند. بخش‌های (1) تا (8) را بنویسید.